

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“Jnana Sangama”, Belgaum -590014, Karnataka.



Full Stack Web Development Report
On

“ATTENDANCE TRACKER”

Submitted by

ROHAN S M (1WN24CS232)
ROMIT P(1WN24CS233)
ROOPESH S(1WN24CS234)
RUDRA (1WN24CS235)

Under the Guidance of

Dr. Priyanka C Hiremath
Assistant Professor

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sept 2025 – Jan 2026

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019**
(Affiliated to Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the project work entitled “**Attendance Tracker**” carried out by Rohan S M(1WN24CS234) Romit P(1WN24CS233) Roopesh S(1WN24CS234) Rudra(1WN24CS235) of 3U who are bonafide students of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-2026. The project report has been approved as it satisfies the academic requirements in respect of **Full Stack Web Development(23CS3AEFWD)** work prescribed for the said degree.

Signature of the Guide
Dr. Priyanka C Hiremath
Assistant Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Kavitha Sooda
Prof & Head of Dept of CSE
BMSCE, Bengaluru

External Viva

Name of the Examiner

Signature with date

1)

2)

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, **Rohan**(1WN24CS232), **Romit**(1WN24CS233), **Roopesh**(1WN24CS234), **Rudra**(1WN24CS235) students of 3rd Semester, B.E, Department of Computer Science and Engineering, BMS College of Engineering, Bangalore, hereby declare that, this Full Stack Web Development entitled "**Attendance Tracker**" has been carried out by us under the guidance of **Dr. Priyanka C Hiremath**, Assistant Professor, Department of CSE, BMS College of Engineering, Bangalore during the academic semester Sept 2025 - Jan 2026.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

ROHAN S M (1WN24CS232)

ROMIT P(1WN24CS233)

ROOPESH S(1WN24CS234)

RUDRA (1WN24CS235)

TABLE OF CONTENTS

Serial No.	TITLE	PAGE NO.
1	Introduction	1
1.1	Overview	1
1.2	Motivation	1
2	Software Requirement Specification	2
2.1	Hardware Requirements	2
2.2	Software Requirements	2
3	ER diagram of the project	3
4	Schema of project	4
5	User Interface Design	5
6	Back End Design	12
7	Conclusion and Future work	14
8	References	15

INTRODUCTION

Attendance management is one of the essential academic activities in any institute to keep track of students' participation and academic regularity. However, traditional methods of attendance are manual, time-consuming, and inefficient. With increased usage of Web technologies, simple, interactive digital attendance system is being felt. The **Attendance Tracker** project is a web application developed using modern frontend technologies with a user-friendly interface for managing and viewing attendance. The system focuses on ease of use, cleanliness of design, and role-based interaction for students and teachers.

1.1 Overview

The **Attendance Tracker** web application is a **frontend-based system** developed using **React** and the **Vite development tool**. The application provides **separate interfaces for Students and Teachers**, allowing each user to interact with the system according to their attendance-related requirements. based strictly on the project files, the key features of the application include **role-based login pages for Students and Teachers**, a **student interface for viewing profile and subject-related information**, and a **teacher dashboard for marking attendance**. The system also supports **reviewing previously recorded attendance details**, enabling better tracking and verification of attendance records. the user interface is designed with a **clean and simple layout**, making use of **reusable React components** to ensure consistency across different pages. Data handling within the application is managed using a local data file (`database.js`), which simulates structured attendance-related data for frontend operations. The project also follows a **responsive and well-organized layout structure**, ensuring better usability across different screen sizes. Overall, the application follows a **modular design approach**, where each feature is implemented as an independent React component. This design makes the system easy to understand, maintain, and modify, and also provides flexibility for future enhancements and expansion.

1.2 Motivation

Many learning institutions are using enterprise-level ERP solutions such as Contineo for managing attendance records. Though such solutions are efficient on a learning institution level, they can be complex, vendor-dependent solutions not geared towards learning. The reason for pursuing this project revolves around designing and building an easy, frontend-based system for tracking attendance that would help students and developers understand how an attendance management system works from within. Moreover, the system is a good platform to apply the latest trends in the frontend part, like component architecture, routing, and UI flow. It is also a scalable platform that can be upgraded in the future by incorporating other services such as backend services, authentication systems, and analytics capabilities.

SOFTWARE REQUIREMENTS

2.1 Hardware Requirements

- A personal computer or laptop with the following minimum specifications or higher:
 - o Intel Core i3 processor or higher
 - o 8 GB RAM (minimum 4 GB supported for basic execution)
 - o 500 GB Hard Disk / SSD storage
 - o Standard keyboard and mouse
- A stable internet connection (2 Mbps or higher) for development tools, libraries, and documentation access

2.2 Software Requirements

- Operating System
 - Windows 10 / 11, Linux (Ubuntu or similar distributions)
- Frontend Technologies
 - HTML5
 - CSS3
 - JavaScript (ES6)
 - React.js
 - Drawio
- Development Environment / IDE
 - Visual Studio Code
- Backend Technologies
 - Node.js
- Server Environment
 - Node.js Runtime
- Package Manager
 - npm (Node Package Manager)
- Web Browser
 - Google Chrome / Mozilla Firefox / Microsoft Edge / Brave
- Version Control
 - Git

3. ER DIAGRAM OF THE PROJECT

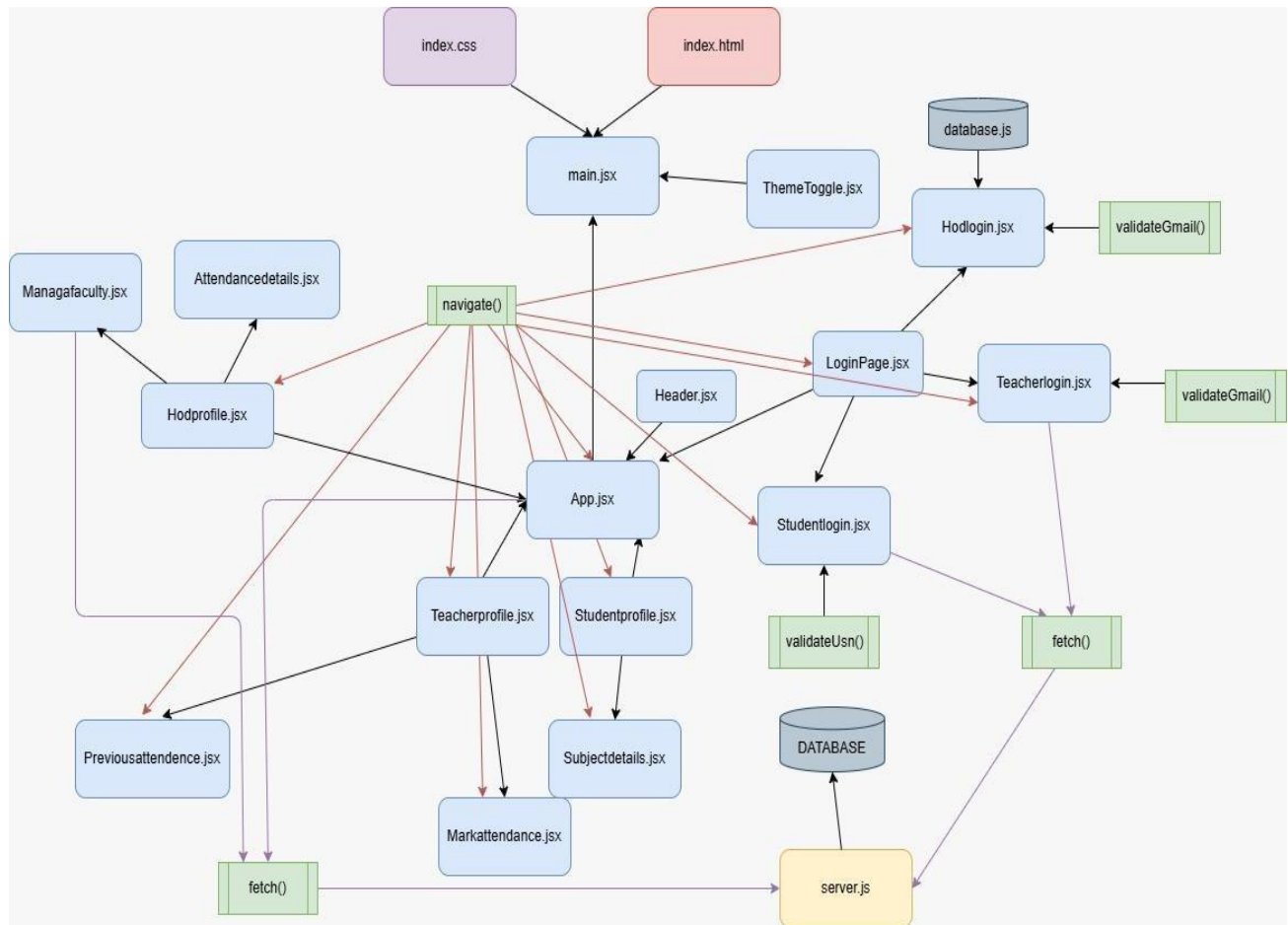


Fig 1. ER Diagram

4. Schema of project

_id	Name	Email	Subject	Classes
-----	------	-------	---------	---------

Table 1. Teacher table

_id	Subject	Credits
-----	---------	---------

Table 2. Subject table

_id	USN	Name	Section	Gender
-----	-----	------	---------	--------

Table 3. Student table

USER INTERFACE DESIGN

There are 4 themes:

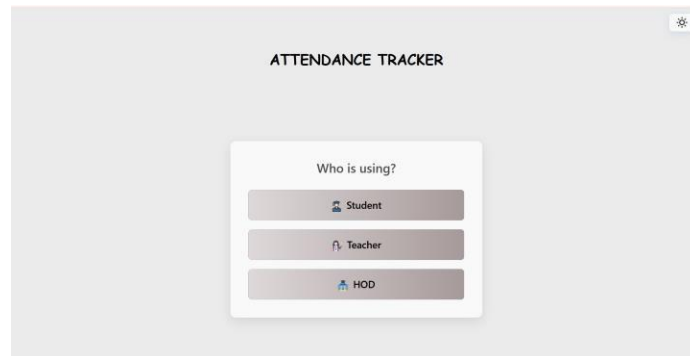


Fig 2. White theme: Black text with white background

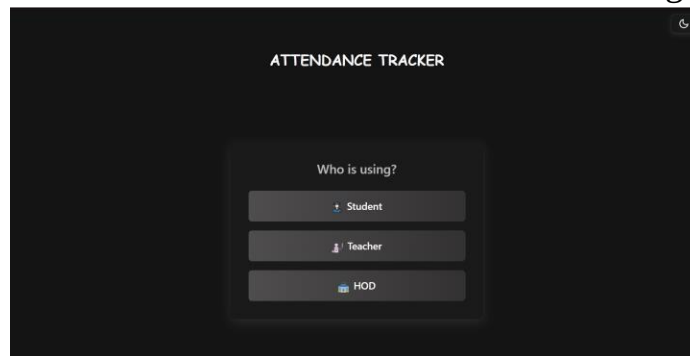


Fig 3. Black theme: White text with black background

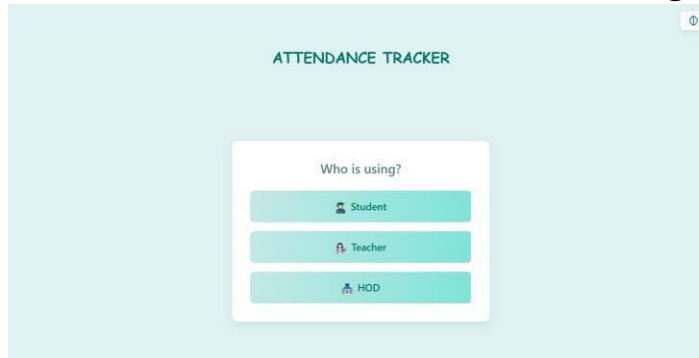


Fig 4. Green theme: Green text with light green background

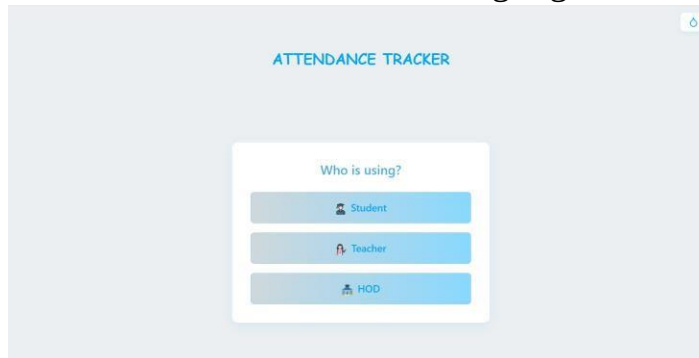
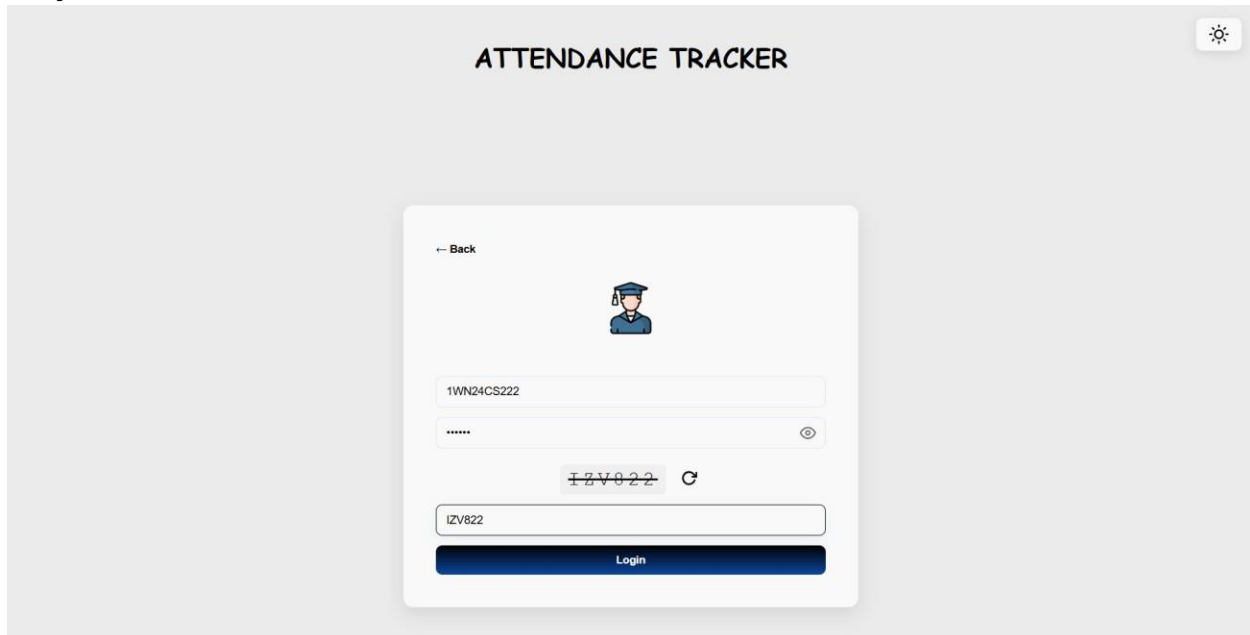


Fig 5. Blue theme: Blue text with grey background

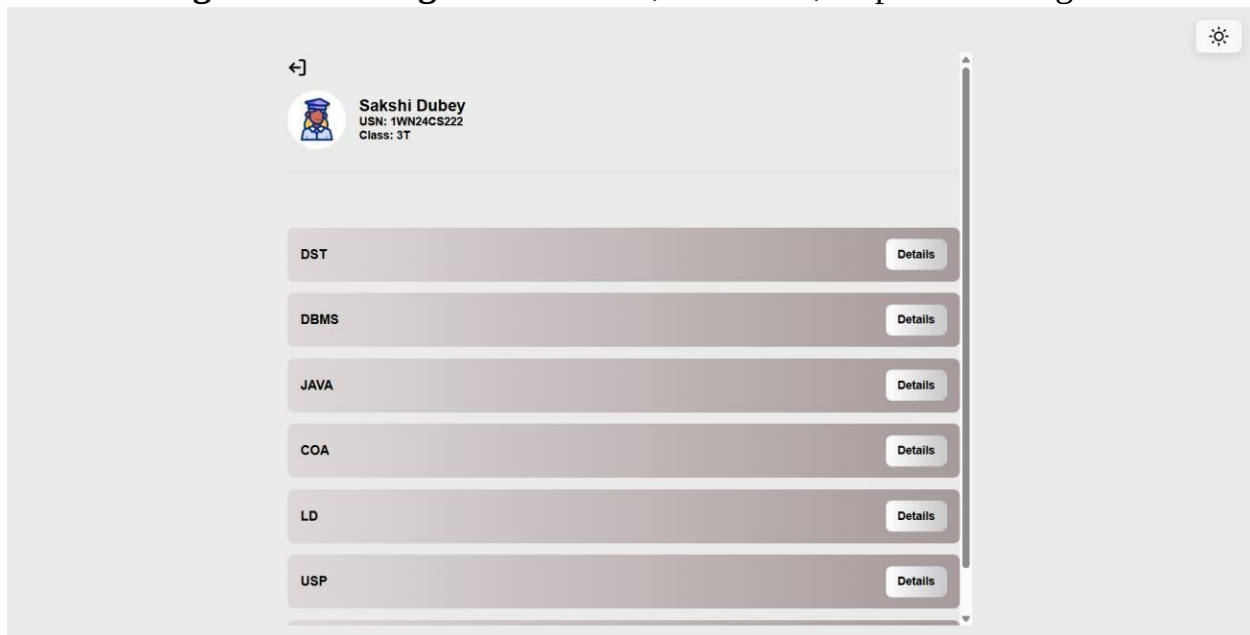
Student view:

Student has to Log in to and can view their attendance in each of the seven subjects.



The screenshot shows a web application titled "ATTENDANCE TRACKER" with a settings icon in the top right. In the center is a login modal with a "← Back" link and a student icon. The modal contains four input fields: a USN field with the value "1WN24CS222", a password field with masked characters "*****" and an eye icon, a captcha field showing "IZV022" with a refresh icon, and a field with the value "IZV/822". A blue "Login" button is at the bottom of the modal.

Fig 6. Student login: Enter USN, Password, Captcha and login



The screenshot shows a student profile page. At the top left is a back arrow icon. The profile header includes a student icon, the name "Sakshi Dubey", the USN "1WN24CS222", and the class "Class: 3T". Below this is a list of seven subjects, each in a grey box with a "Details" button on the right:

Subject	Action
DST	Details
DBMS	Details
JAVA	Details
COA	Details
LD	Details
USP	Details

Fig 7. Student profile: Profile is displayed with student name, USN, Class

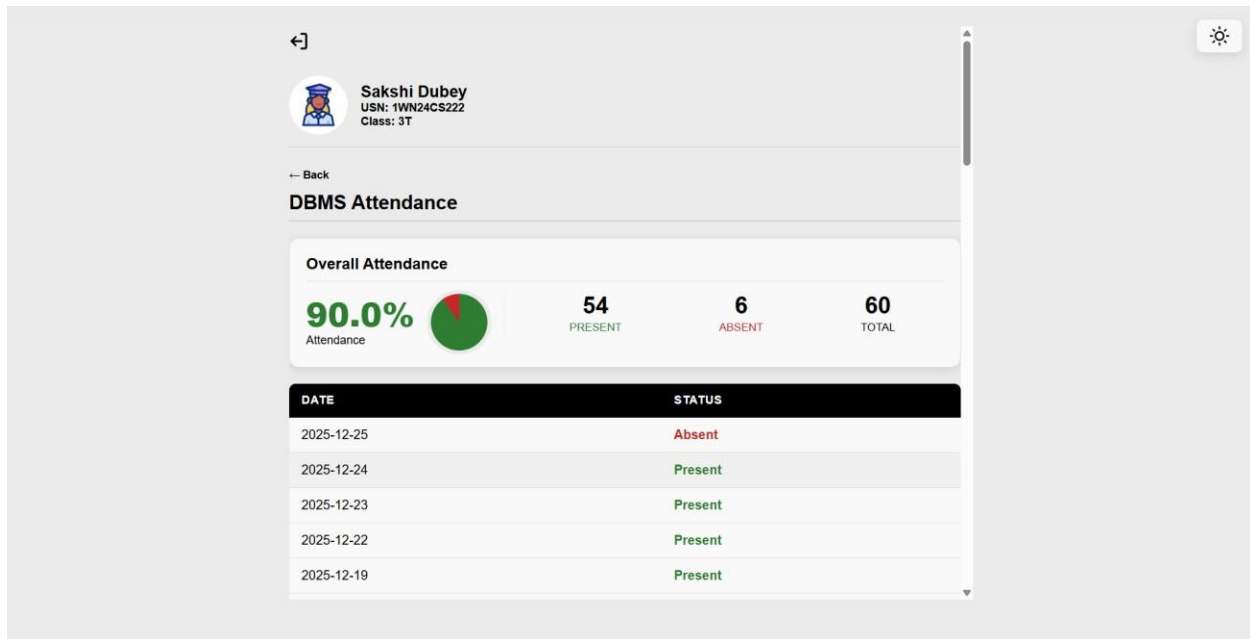


Fig 8. Attendance per subject: Attendance percentage and details for a each subject with pie-chart and attendance status

Teacher view: Teacher has to log in using college email ID and can mark attendance, update previous attendance of students coming under the classes assigned to them.

ATTENDANCE TRACKER

← Back



Fig 9. Teacher login: Enter Email, Password, Captcha

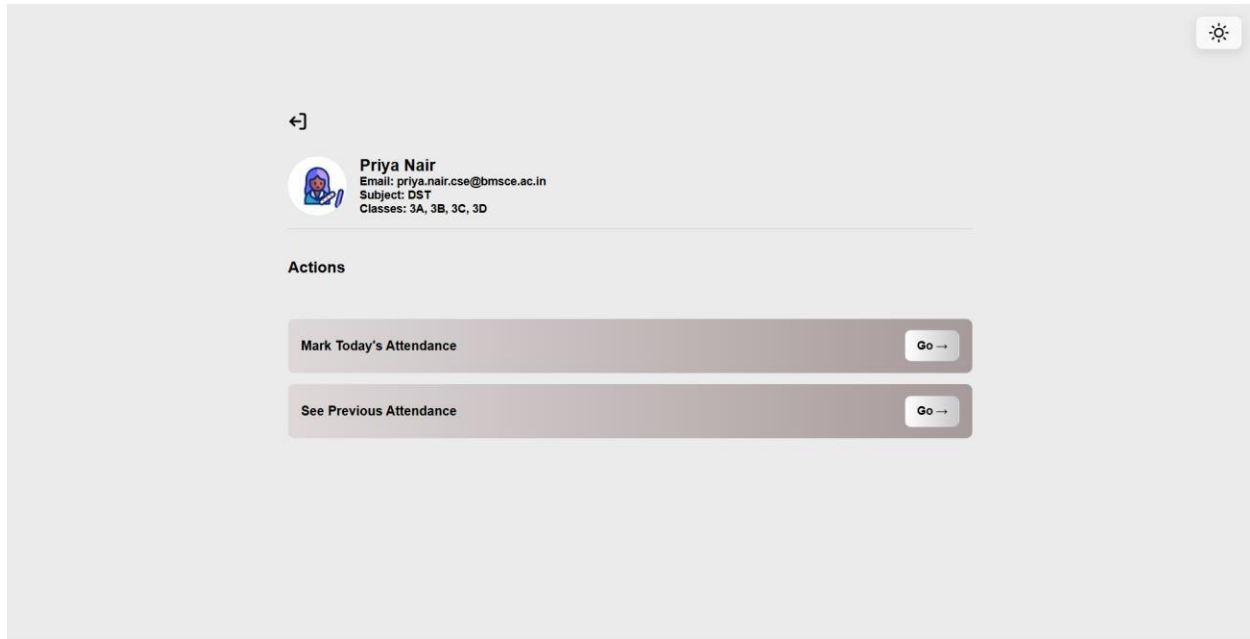


Fig 10. Teacher profile: Teacher profile has name, email, subject and classes assigned.

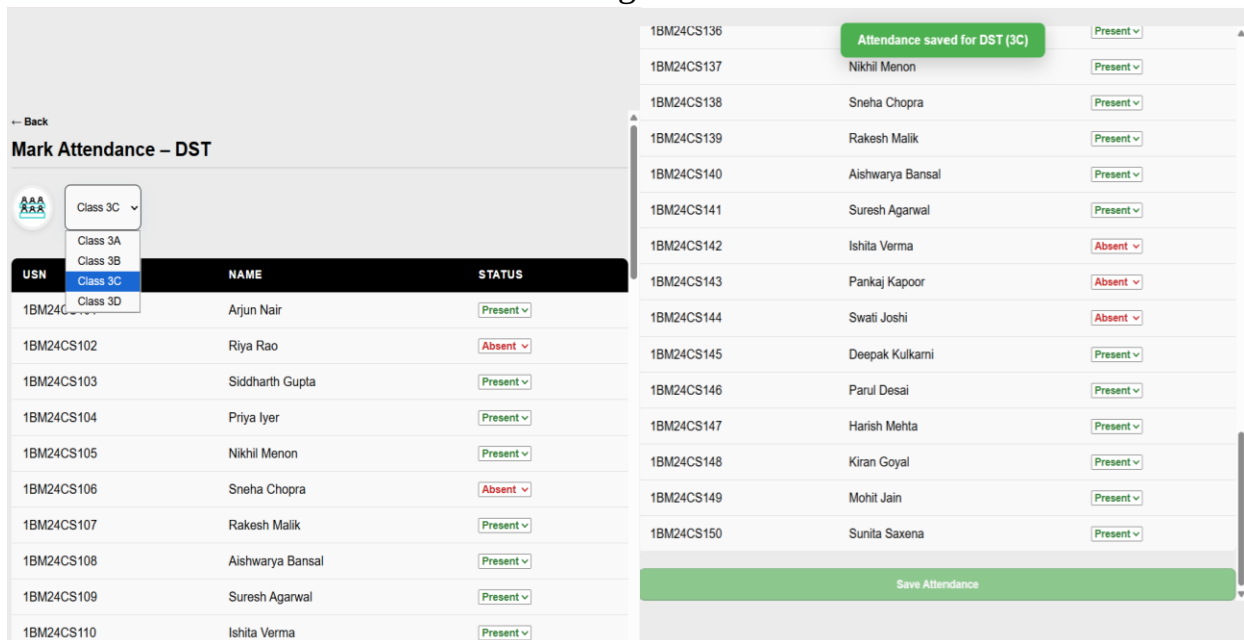


Fig 11. Mark present attendance: Teacher has to select one of the four classes assigned to them and mark absentees for the day

← Back

Previous Attendance – DST

Class 3A

16-12-2025

USN	NAME	STATUS
1BM24CS001	Vivaan Sharma	Absent
1BM24CS002	Divya Reddy	Absent
1BM24CS003	Rohan Patel	Present
1BM24CS004	Neha Singh	Absent
1BM24CS005	Arjun Nair	Present
1BM24CS006	Riya Rao	Present
1BM24CS007	Siddharth Gupta	Absent
1BM24CS008	Priya Iyer	Absent
1BM24CS009	Nikhil Menon	Absent

December, 2025

16-12-2025

Save Changes

Fig 12. Update previous attendance: Teacher has to select the class and date to review the attendance of students and update them when needed

HOD view: Head Of Dept has to login using college email ID and can assign classes to teachers and view attendance of all classes.

ATTENDANCE TRACKER

←

hod.cse@bmsce.ac.in

hPLFsj

hPLFsj

Login

Fig 13. HOD login: Enter Email, Password, Captcha

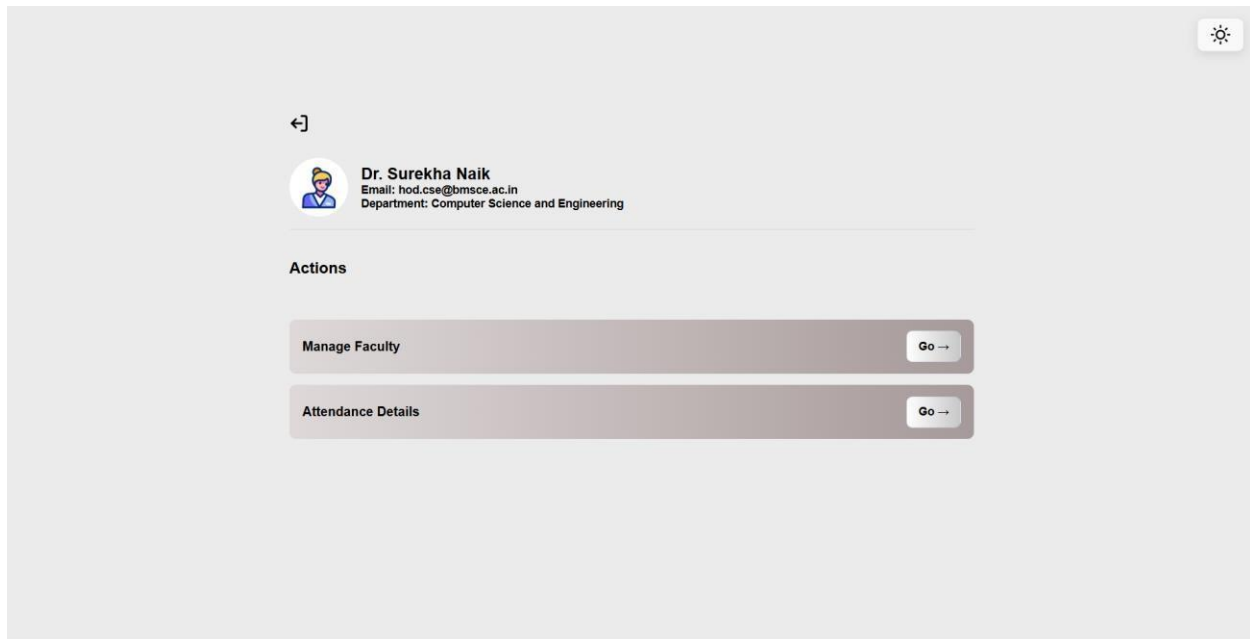


Fig 14. HOD profile: Profile displays HOD name, email, Dept

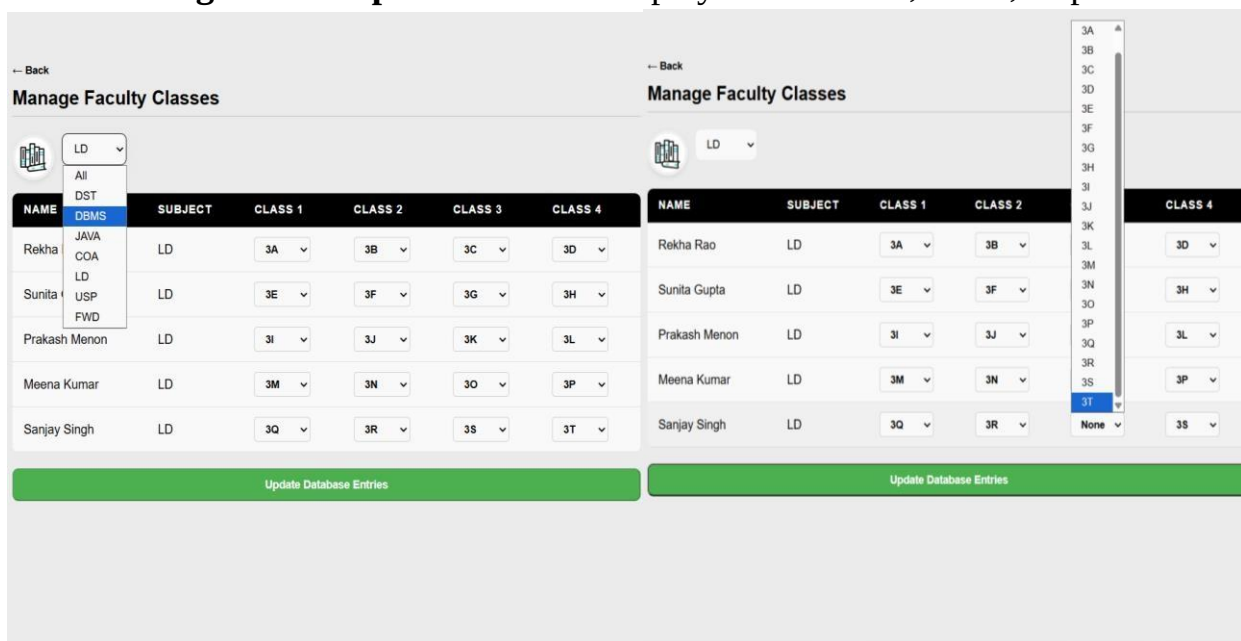


Fig 15. Manage faculty: HOD selects subject and reassign the classes to the faculty

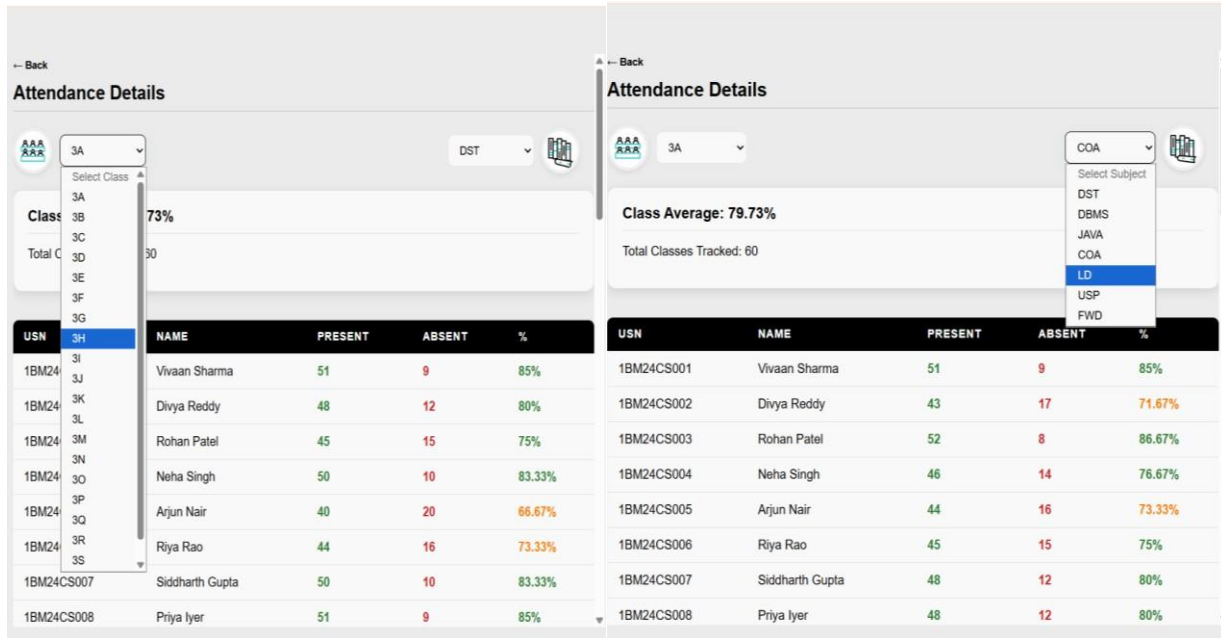


Fig 16. Attendance: Total attendance percentage of students from a class is displayed in correspondence to the subject. Students with attendance less than 75 percentage are highlighted.

BACKEND DESIGN

Import dependencies:

```
1 const express = require("express");
2 const mongoose = require("mongoose");
3 const cors = require("cors");
4 require("dotenv").config({ path: "./.env" });
5
6 const app = express();
7 app.use(cors());
8 app.use(express.json());
9
```

Fig 17. Import express, mongoose, cors, dotenv packages using node package manager.

Routes: URL path that tells the server which piece of code to run when a request arrives.

Student api:

```
1 app.get("/api/students", async (_, res) => {
2   const students = await Student.find({});
3   const out = {};
4   students.forEach(s => {
5     out[s.USN] = {
6       name: s.Name,
7       class: s.Section,
8       gender: s.Gender,
9     };
10  });
11  res.json(out);
12 });
13
14 app.get("/api/students/:usn", async (req, res) => {
15   const usn = req.params.usn.toUpperCase();
16   const student = await Student.findOne({ USN: usn });
17
18   if (!student) return res.status(404).json({ exists: false });
19
20   res.json({
21     exists: true,
22     student: {
23       usn: student.USN,
24       name: student.Name,
25       class: student.Section,
26       gender: student.Gender,
27     },
28   });
29 });
```

Fig 18. These api can be used for accessing student object and validate USN.

Teacher api:

```
1 app.post("/api/teachers/login", async (req, res) => {
2   const email = req.body.email.toLowerCase();
3   const teacher = await Teacher.findOne({ Email: email }).lean();
4
5   if (!teacher) return res.status(404).json({ success: false });
6
7   let formattedClasses = [];
8   if (Array.isArray(teacher.Classes)) {
9     formattedClasses = teacher.Classes.flatMap(cls => cls.split(",").map(s => s.trim()));
10  } else if (typeof teacher.Classes === "string") {
11    formattedClasses = teacher.Classes.split(",").map(s => s.trim());
12  }
13
14  res.json({
15    success: true,
16    teacher: {
17      email: teacher.Email,
18      name: teacher.Name,
19      subject: teacher.Subject,
20      classes: formattedClasses,
21    },
22  });
23 });
24
25 app.get("/api/teachers", async (_, res) => {
26   const teachers = await Teacher.find({});
27   res.json(
28     teachers.map(t => ({
29       email: t.Email,
30       name: t.Name,
31       subject: t.Subject,
32       classes: t.Classes || [],
33     }))
34   );
35 });
36
37 app.put("/api/teachers/:email/classes", async (req, res) => {
38   const { classes } = req.body;
39
40   await Teacher.updateOne(
41     { Email: req.params.email },
42     { $set: { Classes: classes } }
43   );
44
45   res.json({ success: true });
46 });
```

Fig 19. These apis can be used for accessing teacher object and validate emails.

Connect to database:

```
1 mongoose
2 .connect(process.env.MONGO_URI)
3 .then(async () => {
4   console.log("✅ MongoDB Connected");
5   await generateAttendanceFromMongo();
6   app.listen(5000, () =>
7     console.log("🚀 Backend running at http://localhost:5000")
8   );
9 })
10 .catch(console.error);
11
```

Fig 20. Connection to database started using .connect() method.

CONCLUSION AND FUTURE WORK

Conclusion

The Attendance Tracker project provides a systematic and interactive web application designed to simplify and streamline the attendance management process in academic environments. Developed as a React-based application, the system offers role-specific interfaces for students and teachers, ensuring clarity, ease of use, and organized access to attendance information. By adopting a component-based architecture, the application maintains modularity, reusability, and smooth navigation across different screens.

The project serves as a practical demonstration of applying modern frontend development concepts such as state management, routing, and responsive UI design to solve real-world academic challenges. The interactive nature of the system improves user engagement while reducing the complexity commonly associated with traditional attendance systems. Furthermore, the project highlights how a well-designed frontend application can enhance transparency and usability in attendance tracking.

Overall, the Attendance Tracker stands as an effective learning-oriented solution that bridges theoretical knowledge and practical implementation. It not only addresses the academic attendance management problem but also provides a strong foundation for future enhancements, including backend integration, analytics, and mobile support.

Future Work

In the future, the Attendance Tracker system can be enhanced by integrating a backend server and database to support real-time attendance management and permanent data storage. Secure authentication and authorization mechanisms can be implemented to ensure data security and role-based access control. Features such as automated attendance reports, graphical analytics, and notification alerts can be added to improve monitoring, transparency, and decision-making for students, teachers, and administrators.

The scope of the project can be further expanded by introducing mobile application support, enabling users to access attendance information anytime and anywhere. Advanced attendance methods such as QR code-based or biometric systems can be incorporated to increase accuracy and reduce manual effort. Cloud deployment can also be considered to improve scalability, reliability, and accessibility for large academic institutions.

What differentiates this project from existing ERP systems such as **Contineo** is its simplicity, flexibility, and learning-oriented design. While ERP systems are comprehensive, complex, and vendor-controlled, this project is lightweight and fully customizable, making it suitable for academic experimentation and rapid enhancement. The system focuses on clarity, usability, and understanding the internal working of attendance management applications rather than providing a fixed, enterprise-level solution.

REFERENCES

The following websites and online resources were referred to during the design and development of the Attendance Tracker project.

1. W3Schools – HTML, CSS, and JavaScript

W3Schools was used to understand the fundamentals of HTML structure, CSS styling, and JavaScript programming used in the user interface design.

Website:

<https://www.w3schools.com>

[W3Schools Online Web Tutorials](#)

2. JavaScript.info

This website was used to strengthen understanding of JavaScript logic, functions, arrays, and event handling.

Website:

<https://javascript.info>

3. GitHub

GitHub was used to explore sample React projects, understand folder structures, and learn best practices in frontend development.

Website:

<https://github.com>

4. React Official Documentation

The React documentation was referred to for implementing component-based architecture, JSX syntax, props, state handling, and event management used in the project.

Website:

<https://react.dev>

5. Node.js Official Documentation

Node.js documentation was referred to for understanding server-side JavaScript, backend architecture, and REST API concepts.

Website:

<https://nodejs.org>

6. MongoDB Compass Documentation

MongoDB Compass documentation was used to understand visual database management, data inspection, and query execution.

Website:

<https://www.mongodb.com/products/tools/compass>

7. REST API Concepts – MDN Web Docs

MDN Web Docs were referred to for understanding RESTful APIs, request–response cycles, and HTTP methods used in backend communication.

Website:

<https://developer.mozilla.org/en-US/docs/Web/HTTP>

9. Express.js Documentation

Express.js was used as a reference for building lightweight backend services and handling HTTP requests efficiently.

Website:

<https://expressjs.com>